

**Common PS-EC Monitoring SW Guidelines****1 Abstract**

PS-EC Monitoring SW Guidelines enable the exchangeability of diesel and gasoline functions.

Only whole packages can be changed according to the common PS-EC Monitoring SW Guidelines, since the changes are not backward compatible.

PS-EC Monitoring SW Guidelines are base for MDG1.

Changes for MDG1 are part of PS-EC Monitoring SW Guidelines.

2 Table of contents

1	<i>Abstract</i>	<i>1</i>
2	<i>Table of contents.....</i>	<i>1</i>
3	<i>Modification history</i>	<i>3</i>
4	<i>Indication of the FC, whether PS-EC-suitable</i>	<i>5</i>
5	<i>Nomenclature for processes, module IDs,</i>	<i>5</i>
6	<i>Sectioning of the memory area</i>	<i>5</i>
7	<i>PaVaSt creation</i>	<i>6</i>
8	<i>Conversion formula.....</i>	<i>6</i>
9	<i>Labels</i>	<i>8</i>
10	<i>Interface Changes</i>	<i>8</i>
11	<i>Message interfaces.....</i>	<i>9</i>
12	<i>Complement</i>	<i>9</i>
13	<i>Boolean interfaces</i>	<i>9</i>
14	<i>Distribution search</i>	<i>11</i>
15	<i>Header includes</i>	<i>12</i>
16	<i>MoFSrv_Lib and object memory</i>	<i>14</i>
17	<i>Mo_Glbl</i>	<i>15</i>
18	<i>Memory tests</i>	<i>17</i>
19	<i>MoExe_Co</i>	<i>17</i>
20	<i>Compiler as part of the SW-Build.....</i>	<i>18</i>
21	<i>Instruction test.....</i>	<i>19</i>
22	<i>Variant mechanisms for L2 and L3.....</i>	<i>21</i>
22.1	<i>Code words</i>	<i>21</i>



Common PS-EC Monitoring SW Guidelines

22.2	Variant coding	21
22.3	Conditional packaging	22
22.4	Conditional compiling with system constant (variant type C)	22
23	Usage of level 1 system constants as value	23
24	Terminal 15 Turn Off/Turn On without reset	23
25	Measurement points	24
26	Direct hardware access	24
27	Complement Variables	25
28	Request ICO in level 1	25
29	Functional Conventions	26
29.1	Debounce counters	26
29.2	Initial data set	26
30	Link to Checklist	26
31	Validation of conversion to PS-EC Coding Guidelines Monitoring	26
32	SW-Review and FCI	27
33	Recommendation to avoid stack usage	27
34	Software Function Call Coverage	27
35	Extended monitoring task	27
36	Failure Modes of QM-rated services/functions	28
37	Usage of Pointer	29
38	Usage of tool MoFFrm_Nds	292



3 Modification history

Version	Date	Change	Author	Release
1.0	07/02/2012	Initial version for Process Lib	H.Gwinner H.Kutz H.Stadler H.Wolff H.Troyer	released
1.1	23/11/2012	General rework Compiler Version Corrections Request ICO in level 1	H.Troyer	released
1.2	28/01/2013	General rework Compiler Version Mo_Temp	H.Troyer	released
1.3	04/04/2013	Compiler Version	H.Troyer	released
1.4	24/06/2013	Further examples included MDG1 considered Mo_Tmp correction	H.Troyer	released
1.5	22/10/2015	BITUTIL.H deleted Function call coverage considered Usage of stack variables in MDG1 New properties of Mo_Glbl	H.Troyer H.Bien	released

**Common PS-EC Monitoring SW Guidelines**

Version	Date	Change	Author	Release
1.6	29/02/2016	New chapter Extended monitoring task Change of complement handling for counters	H.Bien	released
1.7	14/09/2016	Adaptions regarding autocode generation to match ASCET and MLSL New chapter Failure Modes of QM-rated services/functions	H. Bien Review: H.Troyer H.Wundrack H.Kutz	released
1.8	25/10/2018	Addition of chapter "Usage of pointer" DGS changed to PS-EC	H.Bien Review: H.Troyer H.Gilgenbach H.Wundrack H.Kumar	released
1.9	10/06/2020	Added Chapter "Usage of tool MoFFrm_Nds" The Links were corrected. Update in Chapter 32 "SW-Review and FCI" Update of figure in Chapter 8 "Conversion formula"	Namitha T Review: Shruthi N H.Bien	released

4 Indication of the FC, whether PS-EC-suitable

- FC converted to PS-EC Coding Guidelines have to be marked with DGS-MDG1 in alternative comment of the FC.
- This means that a first PS-EC implementation goes together with a new alternative.
- Definition of PS-EC-suitability: Package can be compiled with PS-EC Make containing gasoline and diesel modules.
- Additional efforts, which are mentioned subsequently, could be necessary for DS- or GS-Make.

Example Alternative Comment in SDOM

Alternative	
Object	FC-ALT : MoFACC_Co / 30.0
Comment	DGS: Daimler projects
Customer	0000 - kein Kundenbezug
ECU gen.	MEDC17
Support state	supported Default value
Disc. exp. date	
ProdLine	MEDC17
Responsibility	
Respons. User	
Release	
Object	FC-REL : MoFACC_Co / 30.0; 0
Comment	DGS: Creation based on the DGS Monitoring Guidelines

5 Nomenclature for processes, module IDs, ...

Specifications:

[Common PS-EC Monitoring naming convention.xls](#)

6 Sectioning of the memory area

Specifications:

Pragma concept is adopted. Memlay macros have to die out.

Consequences for DS and GS:

The Memlay macros are completely used in DS and at L3 in GS. These have to be replaced with the Pragma instructions.

Additional explanations:

Pragma and Memlay macros can exist in parallel.



Common PS-EC Monitoring SW Guidelines

Example how to locate a L2 Code

```
#define START_SECTION_Mo_CodeCyclicCheck
#include "mo_pragma.h"

void MoFESpd_Co_Proc(void)
{
    /* Serve Program Flow Check / Mo
    MoCPFC_Co_Set(MOFESPD_CO_PFC_ID);

    MoCPFC_Co_Check();
}
/* EasyCASE ) */
#define STOP_SECTION_Mo_CodeCyclicCheck
#include "mo_pragma.h"
```

7 PaVaSt creation

Mx17

Specifications:

MSR Procol Block with H_C_FILES generation must be used.
Code syntaxes, addressing schematic and type models are defined in
[Common PS-EC Monitoring MSR PreferenceList.xls](#)

MDG1

Specifications:

MSR Procol Block with H_C_FILES generation must be used.
Code syntaxes, addressing schematic and type models are defined in
[Common PS-EC Monitoring MSR PreferenceList MDG.xls](#)

Example

Services are exported in PaVaSt File because they are scheduled in MoExe_Co.

Short Name	: [MoFTrqLos_Co_Proc]
Category	: STANDARD

Level 1 process scheduled via os_sched.dy

Short Name	: [MoXTrqLos_Co_Proc]
Category	: PROCESS

Important:

Diesel and gasoline code syntaxes, addressing schematic and type models may not be mixed within a function.

8 Conversion formula

Specifications:

Conversion formulas with system constants are not permissible.

Conversion formulas are all maintained in the central conversion formula OSP. In case of new conversion formula, first check whether a suitable conversion formula is available. If not, then a new conversion formula has to be requested through the clearing.



Common PS-EC Monitoring SW Guidelines

No new Mo conversion formula must be created for PS-EC packages.

Generally, no Mo conversion formulas are used for new PS-EC interfaces. When applying PS-EC Coding Guidelines Mo conversion formulas have to be replaced, if corresponding platform conversion formulas exist. (Quantization must not be changed)

Consequences for DS and GS:

All new conversion formulas have to be cleared with the CEL package responsible.

When applying PS-EC Monitoring SW Coding Guidelines remove conversion formulas from MO_GLBL.

If neither a Mo conversion formula nor corresponding platform conversion formula does exist in CEL package, a new platform conversion formula has to be cleared with the CEL package responsible.

[Example: How to request a computation method](#)

PS-EC Architecture Clearing Centre

<https://inside-docupedia.bosch.com/confluence/display/dgsswarchitecture/DGS+Architecture+Clearing+Centre>

Computation Methods



[Request new \(arithmetic\) compu methods \(cat. RAT_FUNC\)](#)

[Request new verbale compu methods \(category TAB_VERB\)](#)

[Question about computation methods](#)

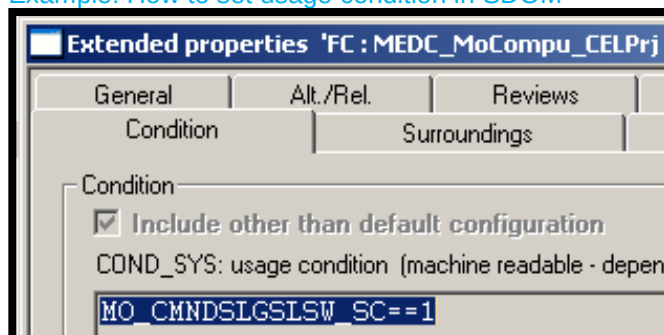
DS (Clearcase): [Request new compu methods](#)

GS (Nestor): [Request new compu methods](#)

Clearing of conflicts:

- Explicit PS-EC conversion formula for CEL package, i.e. not for all projects.
In all FC : MEDC_MoCompu should be entered as **usage condition** MO_CMNDSLGSLSW_SC==1. (Info block of the FC).

[Example: How to set usage condition in SDOM](#)



In all FC : Mo_Glbl, which still contain Monitoring conversion formula (i.e. are not PS-EC Mo_Glbl) must be entered as **usage condition** MO_CMNDSLGSLSW_SC==0.



Common PS-EC Monitoring SW Guidelines

For all diesel packages which have not yet applied the **usage condition** **MO_CMNDLGSLSW_SC==0** on Mo_Glbl, a manual check has to be done.

Note: The usage condition can be entered in the info block in the tab condition as long as the FC is open. If required, a revision should be created.

- No system constant dependent Mo conversion formula shall be migrated to CEL package FC : MEDC_MoCompu . System constants have to be replaced by fixed values.
- Consistency should be ensured in the CEL package.

9 Labels

Specifications:

Interface test in the PaVaSt File is mandatory as soon as PS-EC Monitoring SW Coding Guidelines are applied. This means that variable/parameter definition (e.g. name, conversion formula, data type, array size) must be specified for imported variables/parameters.

Example: Necessary interface tags for DAMOS consistency check

Import Interface for variables and parameters	
In order to detect interface problems as early as possible, more as the mandatory required information shall be specified when importing an item in a PaVaSt-file. This enables DAMOS++ to perform a consistency check between the exporting and importing functional component. Sections for each data dictionary element to be compared are listed in the following.	
Table 1: Additional elements for imported variables and parameters	
Imported bit	Simple models like TYP_SingleBit*, TYP_MessageBit* ²⁾
Imported message	Base type Computation method ²⁾ Data constraint ³⁾ Array size ⁴⁾ SW-Implementation policy ²⁾
Imported variable	Base type Computation method ²⁾ Data constraint ³⁾ Array size ⁴⁾
Imported (adj.) system constant	Base type Computation method ²⁾ Data constraint ³⁾
Imported calibration parameter	Base type Computation method ²⁾ Data constraint ³⁾ Array size ⁴⁾
Imported calibration curve, map	Base type Record layout Computation method (input,x,y, output) ²⁾ Data constraint ³⁾ Array size (GMAP, GCUR) ⁶⁾

10 Interface Changes

In general change of an implementation of an interface is not allowed.
Especially DS states Mo*_st* should not be changed from category "VALUE" to "BIT".
Mo_stlCOMsg is kept as value and uint8 Message as in Diesel systems.



11 Message interfaces

Interfaces within L2 and L3 may not be implemented as messages. Interfaces to the L1 and generally for MoX should be implemented as messages.

State DS: Done.

State GS: Change is done package-wise when PS-EC Monitoring Coding Guidelines are applied. The interfaces are not renamed.

Messages from L2 and L3 must end with Msg in the description. This is not applicable for L1.

12 Complement

Complement variables must be written from the same source as the corresponding original monitoring variable. Therefore a temporary variable (Mo_Temp or stack variable (only for MDG1)) has to be used to store a calculation before writing the result into the monitoring variable as well as the complement variable.

13 Boolean interfaces

Boolean interfaces within L2 and L3 are implemented with monitoring-specific bit macros (DAMOS based). Bit macros exist for cyclically checked memory area with complement, and for non-cyclically checked memory area (instruction test and L3). Bit access is realized with <Bit name>_MoGet, _MoSet, _MoClr, _MoPut Bit macros.

Example: Boolean Interface for L2 and L3 old/new

<pre>MoF_stStrt = MOF_STRTENBL_MSK; MoF_stStrtCpl = ~MoF_stStrt;</pre>	<pre>MoF_bStrtEnbl_MoSet;</pre>
---	---------------------------------

Complements are updated only at the respective Bit position. Complement check is done bitwise with the help of a new DAMOS Bit macro <Bit name>_MoCplChk

Example: Complement check old/new

<pre>if ((MoF_stStrtCpl != (uint8)~MoF_stStrt)</pre>	<pre>if ((MoF_bStrtEnbl_MoCplChk) && (MoF_bPTActv_MoCplChk)</pre>
--	---

Naming convention for Bit base:

<Id>_bb<Out_Loc>[<DdMem>][<DdTk>][n]

<Id>	Full name of component without underscore, e.g. MoFTrqPtdMax
<Out_Loc>	“Out“for global/exported Bits, “Loc“ for local Bits
<DdMem>	NoRelni, Relni, Nov (NonVolatile), Prot (Protected)
<DdTk>	Cooperative Task: “Coop“, preemptive Task: “10ms“, “20ms“, “Syn“, etc.
<n>	ordinal number, if there is not only one Bit base in FC

Further Example:

MoFACCCo_bbOut1

MoFTrqPtdMax_bbLocRelni2



Common PS-EC Monitoring SW Guidelines

```

Available Bitmacros:
MoFACC_flgPtd_MoSet; MoFACC_flgPtd_MoClr;
MoFACC_flgPtd_MoPut; MoFACC_flgPtd_MoGet;
MoFACC_flgPtd_MoCplChk

MoFACC_flgPtd                                /* Example
Packed Bit */
MoFACCco_bbOut1, MoFACCco_bbOut1Cpl          /* Example Bitbase
and Complement */
TYP_Mo_PackedBitCyclicCheck                  /* Typmodell for
Packed Bit */
TYP_Mo_RamBbCyclicCheckU8                    /* Typmodell for
Bitbase */
TYP_Mo_RamBbCplU8                            /* Typmodell for
Bitbase Complement */

Initialisation: Complement of bitbases are always initialised first;
bitpositions only if necessary.
MoFACCco_bbOut1Cpl = ~0;                      /* Step1:
Initialisation of bitbase complement */
MoFACC_flgPtd_MoSet;                          /* Step 2:
Additionally, if Bitposition is initialised != 0 */
Complement Check: Only Bitpositions are checked; bitbase not.
MoFACC_flgPtd_MoCplChk                      /* Complement
with all used Bits */

```

The new procedure corresponding to PS-EC coding guidelines cannot be used within monitoring, but the procedure with packed bits is used according to the previous GS procedure.

Reason:

Object neutrality, complement double storage.

It is accepted that the value of the constant for the bit position changes. In the reading function the bit position does not have to be known. This is handled by DAMOS.

Exceptions: Mo_stMoF, Mo_stMoC and Mo_stErrHyb. These contain bits that are written by several functions and should be handled as before.

Complement is written from the value as until now in MEDC17.

Overlay bits are defined in Mo_Glbl.

MoCMem_st and corresponding masks are defined in MoCMem_Co for the new PS-EC memory checks.

Example: Existing boolean Interface for L2 and L3 written by several functions

```

SW Variable      :
Short Name       : [Mo_stMoC]
Category        : VALUE
SW Class Ref     : (TYP_Mo_RamCyclicCheckU8)

SW Variable      :
Short Name       : [Mo_stMoCCpl]
Category        : VALUE
SW Class Ref     : (TYP_Mo_RamCpl8)

SW Variable      :
Short Name       : [Mo_bCplChk]
Category        : BIT
SW Bit Representation
  Bit Position   : 0
SW Class Ref     : (TYP_B_Overlay)
SW Host Variable :
  SW Variable Ref : (Mo_stMoC)

```



Common PS-EC Monitoring SW Guidelines

Owned variables:

```
SW Variable Ref.      : (Mo_stMoC)
SW Variable Ref.      : (Mo_stMoCCpl)
SW Variable Ref.      : (Mo_bCplChk)
```

Exported variables:

```
SW Variable Ref.      : (Mo_stMoC)
SW Variable Ref.      : (Mo_stMoCCpl)
```

MoGbl Header

```
[Mo_stMoC] - Status aus Ebene 3
#define MO_CPLCHK_MSK      0x01
#define MO_RAMERR_MSK     0x02
#define MO_ROMERR_MSK     0x04
```

No backward compatibility is possible.

In order to decouple implementation of PS-EC Monitoring SW Coding Guidelines and functional unification following approach has been taken:

GS

All Bit bases and Bits are removed from Mo_Glbl except the above.

Bit names Mo_ and MoF_ will not be changed, but defined in the writing functions.

Only for Mo_ Bits new Bit bases have to be created in the writing functions.

For MoF Bits existing Bit bases can be kept.

DS

Existing Bit bases and Bit positions have to be kept as long as the Spec remains unchanged.

When Spec is changed only the Bits will be shown and Bit bases and Bit positions can be changed then.

For new Boolean interfaces to L1, diagnosis parts in L2 and L3 and within MoX, the general L1 PS-EC Coding Guidelines has to be applied.

The naming convention for Diagnostic Bits is the following Mo_flgIrrvErr , Mo_flgTest..

General Coding Standard en

Only TRUE/FALSE should be used at assignments of boolean variables. Monitoring specific defines like MO_TRUE, MO_FALSE ... are obsolete

[Example: Boolean Interface for L2 and L3 old/new](#)

```
Query for FALSE if (!test_b) /* preferred */
if (test_b == FALSE)
Query for TRUE if (test_b) /* preferred */
if (test_b != FALSE)
```

14 Distribution search

GS usually operates with 8 axis points.

Whereas DS defines the number of axis points with system constants in Mo_AxisPoints (Without a change of initial calibration values is only a reduction of axis points possible).

**Common PS-EC Monitoring SW Guidelines**

PS-EC:

Axis points are defined in PaVaSt of each FC. System constants are not used anymore.

Manual coded distribution searches and code generated distribution searches use the same naming convention.

But the generated name of the distribution search result differs from manual Code

⇒ Import of distribution search in PaVaSt

Short term solution:

To avoid problems, distribution search result must be deleted from the generated PaVaSt File. For import only the distribution search is necessary.

Long term solution:

To ensure portability between manual code and autocode generation for monitoring, the naming convention for the result of a distribution search must be changed.

Naming convention for result of a distribution search:

<Name of distribution search>_result

Example: Distribution search in PaVaSt File

SW Variable	:
Short Name	: [MoFESpd_nEng_Ax_result]
Category	: VALUE
SW Class Ref	: (TYP_Mo_RamCyclicCheckS32)
SW Variable	:
Short Name	: [MoFESpd_nEng_Ax_result_Cpl]
Category	: VALUE
SW Class Ref	: (TYP_Mo_RamCpl32)
SW Cal Param	:
Short Name	: [MoFESpd_nEng_Ax]
Category	: AXIS_VALUES
SW Addr Method Ref	: (Mo_DataCyclicCheckByte)
SW Base Type Ref	: (uint8)
SW Calibration Access	: READ-WRITE
SW Cal. Param Axis Set	
SW Axis Index	: 1
SW Axis Individual	
SW Variable Refs	
SW Variable Ref.	: (MoFESpd_nEng)
SW Max Axis Points	: 8
SW Min Axis Points	: 8
Cal. Param Target	
SW Variable Ref	: (MoFESpd_nEng_Ax_result)
SW Code Syntax Ref	: (Mo_DataAxisUS)
SW Record Layout Ref	: (Mo_DataAxisUS)

15 Header includes

Specifications:

1.

Global header is Common.h.

COMMON.H must be included in each FC.

COMMON.H must be placed as first Include, because some other Level 1 headers need defines from COMMON.H

Monitoring Headers should be placed after Common.h, level 1 headers. Because if there would be the same define in a level 1 header and in a monitoring header, define of monitoring header wins.



Common PS-EC Monitoring SW Guidelines

Example: Header Include

```

#include    "common.h"
#include    "reset.h"
#include    "dsm.h"
#include    "mo_glbl.h"
#include    "mofvar_msg.h"

#include    "mofvar_co_dat.h"
/*

```

2.

Interfaces are generally to be implemented in the PaVaSt file.

The _dat.h header is included in the C-file. This means entry H_C_FILES in MSR Protocol Block.

Example: Tag for _dat.h generation in PaVaSt File

```

SW Collection      :
Short Name         : [H_C_FILES]
Category           : PROCOL_COMP

```

Only one header per FC should be used.

External declarations of SW services necessary from L1. E.g. Core Header is directly included in the respective C-file.

Necessary header:

Mo_Glbl Header

MoExe_Co.h and MoFSrv_Lib.h are included through Mo_Glbl.h

(Remove redundant header from the C-files)

MoExe_Co Header contains:

- IDs,
- expected values for partial response contribution
- FIT
- PFC
- External declaration of the SW Services Standard that is called through MoExe_Co.
- External declarations of the SW Services Standard in level 3 which are used in level 2 e.g. **external void MoCCPU_Co_SeqStrt** are imported via mocfc_pub.h and if necessary via mocmm_pub.h.

Reason for external declarations:

In future, SW Services and their external declarations are provided through DAMOS (EOS). Then the external declarations of the SW Services Standard can be simply removed.

3.

Interfaces to L1.

mo_pub.h in the FC MoExe_Co. External declarations of stable, not project specific SW Services are centrally made available to L1. E.g. from the memory tests.

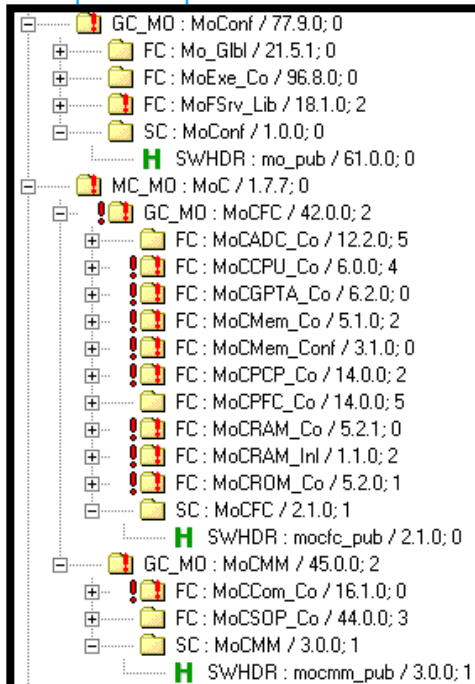
Hint: Existing header includes in Core must be replaced with mo_pub.h

Mo_pub.h is placed in self component of MoConf. Mo_pub.h includes GC specific mocfc_pub.h. If necessary other GC specific Headers could be added.



Common PS-EC Monitoring SW Guidelines

Example: mo...pub.h in SDOM



Msg.h Header

With H_C_FILE Generation the Msg.h header has to be included in the C-File

4.

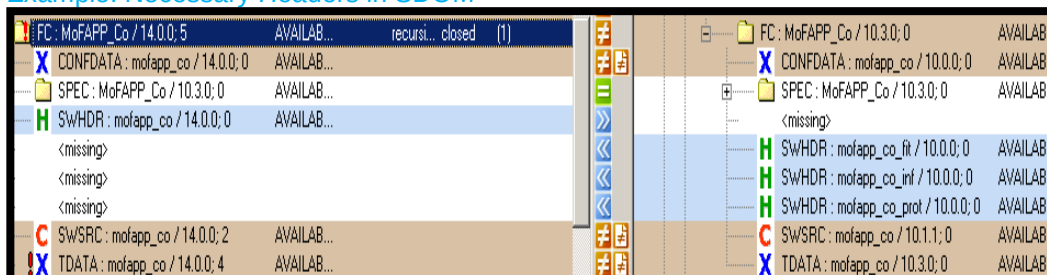
Local Definitions.

<FC>.h at the most only one header is permissible.

Content of Header File:

- Instruction test response contributions
- Content from former _prot.h, _pub.h, _inf.h, etc. which are not supported by PaVaSt File are included in C-File(local constants,...). Exceptions which have to remain in Header File: complex structures, because they cannot be created in PaVaSt File.

Example: Necessary Headers in SDOM



PS-EC Header file concept is replaced by above rules for Monitoring.

16 MoFSrv_Lib and object memory

MoFSrv_Lib 9.0.0 is first PS-EC unified version

Specifications:



Common PS-EC Monitoring SW Guidelines

1. MoFSrv_Lib 9.0.0 is backward compatible and can be used with DAMOS
OSP BTC: DatGenDGS / 2.7.5 and later versions for GS PVER and
DatGenDGS / 3.0.5 and later versions for PS-EC PVERs
2. Preferred solution for macro calls with or without parameter such as:
MO[G]K[F,L][_P]<label>[{(x, y), (x)}]
=> The gasoline and diesel world continues to be supported.
Diesel directly calls the services with parameter.
Gasoline uses the DAMOS macros. There, the parameter is encapsulated
through DAMOS macros.
Unified solution is to use the DAMOS macros (like GS standard).
Advantage: Less sensible to errors during coding.

Example: How to specify a map in PaVaSt

MoFDrDem_rAPP = MO_KF_MoFDrDem_rAPP1_MAP;	
SW Cal Param	:
Short Name	: [MoFDrDem_rAPP1_MAP]
Category	: MAP_INDIVIDUAL
SW Addr Method Ref	: (Mo_DataCyclicCheckByte)
SW Base Type Ref	: (uint8)
SW Calibration Access	: READ-WRITE
SW Cal. Param Axis Set	
SW Axis Index	: 1
SW Axis Individual	
SW Variable Refs	
SW Variable Ref.	: (MoFAPP_u)
SW Max Axis Points	: 8
SW Axis Index	: 2
SW Axis Individual	
SW Variable Refs	
SW Variable Ref.	: (MoFDrDem_rAADVarKL)
SW Max Axis Points	: 8
SW Code Syntax Ref	: (Mo_DataMapWU8)
SW Compu. Method Ref	: (Mo_r_prc_qOp39)
SW Record Layout Ref	: (Mo_DataMapWU8)

17 Mo_Glbl

Mo_Glbl 21.0.0 is first unified PS-EC version.

Specifications:

Mo_Temp variables are extended to 20 * 32 Bit Array.

Mo_Temp variables are tested in a cyclic manner.

Mo_Temp variables may be used only in functions scheduled by MoExe_Co_Proc

MDG1:

With ECU-Generation MDG1, Level3 introduced a stack write ability test. Therefore it is allowed to use stack variables in monitoring functions.

Mo_Tmp variables can still be used as well.

Reverse compatibility of Level2 functions to Mx17 must be ensured, if the function shall be used in Mx17 projects as well. In this case Mo_Tmp variables are mandatory.

Mx17:

In Mx17 there is no stack write ability test present, therefore Mo_Tmp variables are mandatory.

Definition of Mo_Tmp:



Common PS-EC Monitoring SW Guidelines

Unions are not ANSI C compatible.

Therefore Mo_Tmp union from Mx17 must be replaced by Mo_Tmp structure for MDG1.

In consequence RAM consumption increases by factor 7 compared to a union in Mx17, because in a union the different structure elements overlay.

The different structure elements of Mo_Tmp can be accessed identically in Mx17 and in MDG1.

Example: Mo_Tmp Mx17 and MDG1

```
typedef union
{
    uint32* p32;
    uint32  u32;
    sint32  s32;
    struct {sint16 l; sint16 h;} s16;
    struct {uint16 l; uint16 h;} u16;
    struct {uint8 ll; uint8 lh; uint8 hl; uint8 hh;} u8;
    struct {sint8 ll; sint8 lh; sint8 hl; sint8 hh;} s8;
} Mo_Tmp32_t;

typedef struct
{
    uint32* p32;
    uint32  u32;
    sint32  s32;
    struct {sint16 l; sint16 h;} s16;
    struct {uint16 l; uint16 h;} u16;
    struct {uint8 ll; uint8 lh; uint8 hl; uint8 hh;} u8;
    struct {sint8 ll; sint8 lh; sint8 hl; sint8 hh;} s8;
} Mo_Tmp32_t;
```

Precondition:

Multiply use of Mo_Tmp variables with different data types inside a specific function, as it is furthermore possible for Mx17 with Mo_Tmp union, must be forbidden for functions which are compatible for Mx17 and for MDG1 as well.

Conclusion:

- 1) Multiply use of Mo_Tmp variables with different data types inside a specific function is from now on forbidden for Mx17 and for MDG1 as well.
- 2) If a function in Mx17 is overworked, a change according 1) is only recommended for bigger changes and must be handled very carefully.
- 3) Functions which are migrated to MDG1, changes must be done according 1).

Examples:

Mo_Tmp[0].u8.ll, Mo_Tmp[0].u8.lh, Mo_Tmp[0].u16.h is allowed.

Mo_Tmp[0].u8.ll, Mo_Tmp[0].u8.lh, Mo_Tmp[0].u16.l is forbidden.

Mo_Tmp[0].u32 is allowed, but no other structure element could be used.

Example: Forbidden usage of Mo_Tmp

```
MonTmp_tqEngMinH_T0u8hl = MOFFrm_Mirr_Buffer[MOFACC_CANBYTE3] & MOFACC_0x1F;
MonTmp_tqEngMinL_T0u8ll = MOFFrm_Mirr_Buffer[MOFACC_CANBYTE2];

MOFACC_flgECUErr_MoPut(MonTmp_tqEngMin_T0s16l == MOFACC_0x1FFF);
```

Mo_ctRst and complement is defined in Mo_Glbl.

EEPUpdt Proc is replaced by Inline function in MoFSrv_Lib(MoFSrv_Lib_cntr_Max...).
Actual in MoFSrv_Lib 9.0.0

Example: Service to store e.g. max . value of a counter



```
MoFSrv_Lib_cntrMax_U8U8(&MoFESpd_cntrMaxNotVlt1U8, MoFESpd_ctDeb, Mo_swtClrCntrMax_C);
```

MO_ADDCONPFC_MSK is shifted to the bit position 7 during the conversion to PS-EC Monitoring SW Coding Guidelines.

Test required if the mask MO_ADDCONPFC_MSK is always used.

Mo_Glbl Confdata - SW Reset LoopCondition

DS

SWRESET_MOC SOP_UVL_COND	Reset cycle time 0, maxnum 3
SWRESET_MOC_COND	Reset cycle time 2s, maxnum 3
SWRESET_MOF_COND	Reset cycle time 2s, maxnum 8

GS

SWRESET_PREICO_COND	Reset cycle time 0, maxnum 8
---------------------	------------------------------

PS-EC

A first implementation contains superset of DS and GS.

FC: Mo_Glbl / 97.*.* solely for MDG

From FC: Mo_Glbl / 97.6.0; 1 on the following features are introduced compared to previous versions:

- Mo_Glbl is part of the PFC
- Initialization and re-initialization of Mo_ctRst
- Complement check of:
 - Mo_ctRst
 - Mo_stMoF
 - Mo_stErrHyb
- POSTDRIVE treatment of Mo_ctRst is taken over from MoFICO_Co

18 Memory tests

Specifications:

The new memory tests can only be used with the PS-EC make.

Additional explanations:

DS can remain with the old DS memory test till it is released for DS-PST. GS must use the new PS-EC-memory test for PS-EC make.

19 MoExe_Co

Specifications:

1.

If conditional compiling with system constant MOCCPU_IT_SY is used in MoExe_Co, emFace must include the check of this system constant (Partial configuration Mo-Conf).

Additionally make must break if no allowed value is set for MOCCPU_IT_SY.

**Common PS-EC Monitoring SW Guidelines**

Example: How to check if system constant is present

```
#ifndef MOCCPU_IT_SY
#error >>> 'MOCCPU_IT_SY' undefined!
#endif
```

Recommended is to create MoExe_Co without system constant.

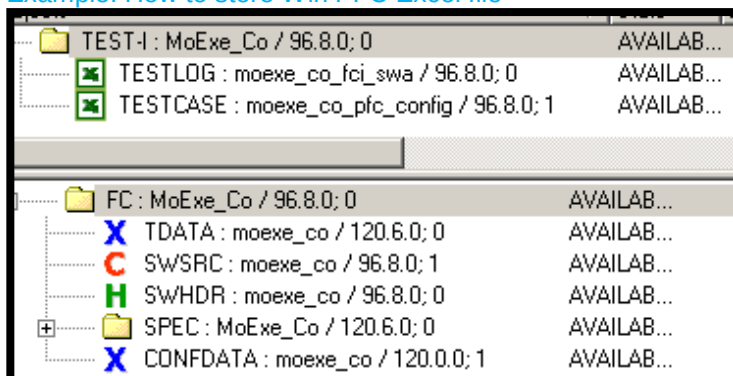
2.

Win PFC Excel tool must be used.

[Template PFC Config MoExe DS GS PS cv.xls](#)

Corresponding excel sheets must be stored in specific MoExe_Co FCI Test container.

Example: How to store Win PFC Excel file



3.

Define Customer specific process order and document in a MoExe_Co Template. Change of process order only for greater changes or necessary for functional unification.

4.

Diagnosis functions are called in MoExe_Co_Diag_Proc

Example: MoExe_Co diagnosis process

```
void MoExe_Co_Diag_Proc(void)
{
    if (SyC_stSub == SYC_DRIVE)
    {
        switch ((uint8)((uint8)(MoExe_ctCyc - 1) & (uint8)MOEXE_CT_MSK))
        {
            case MOEXE_SLOT_A_MSK:
```

5.

For L2 processes prototypes must be available through external instructions in the MoExe header.

20 Compiler as part of the SW-Build

For monitoring there is no specific Compiler version and Compiler options necessary anymore. A released SW Build tool chain has to be used.

Preliminary decision regarding compiler:

**Common PS-EC Monitoring SW Guidelines**

folgende Strategie ist in gemeinsamer Absprache zwischen EPQ/ESB/ESS und ISO26262-Team getroffen worden.

- 1) freigegebene Make-/Build-Tools (Tool-Kette) kommt immer von der Toolentwicklung
- 2) für Safety-Umfänge wird es bei DGS keine separate Tool-Version (inkl. Schalter/Optionen) mehr geben
- 3) Einsatz freigegebener Tool-Versionen wird in der QGC4 für jedes Projekt abgefragt
- 4) falls Projekt keine freigegebene Tool-Version einsetzt, muß Projekt durch Sondermaßnahmen (z.B. Einsatz eines SCAN-Tool für fehlerhafte Compiler-Versionen) die Serientauglichkeit nachweisen.
- 5) allgemeines Toolqualifizierungsvorgehen um 1) u. 2) zu ermöglichen ist aktuell im Review im DS/GS/DGS-Safety-Team -> Hr. Pitzal ist hier zusammen mit dem PR für Toolqualifizierung aktiv (Dokument folgt noch)
- 6) Vorgehen gilt für alle SG-Generationen und alle Make-/Build-Prozesse

Konsequenz für uns:

Eine harte Abprüfung einer speziellen Compilerversion bzw. -optionen im eMFace-/UEKO-Check ist nicht mehr notwendig.
Hinweis an Projekt, daß gemäß 1) bzw. 4) eine freigegebene Toolkette oder gemäß 5) eine Sondermaßnahme zu verwenden ist, | kann natürlich nicht schaden.

21 Instruction test

In Mo_Glbl, a set of generic variables is created for the instruction test. They are redefined in L2' to represent L2 variables and L1 messages (only imported).

Generic variables are exchanged through PaVaSt interface.

E.g. Mo_data00ItU8, Mo_flg0ItMsg, Mo_data00ItU32Msg, Mo_data00ItAryS16

Example: Import of generic variables for FIT in PaVaSt

```

SW Variable Ref. Sys Cond
SW Variable Ref      : (MoCCPU_Query)
SW Sys Cond          : (i(MOCCPU_IT_SY) ==1)
SW Variable Ref. Sys Cond
SW Variable Ref      : (Mo_flg0It)
SW Sys Cond          : (i(MOCCPU_IT_SY) ==1)
SW Variable Ref. Sys Cond
SW Variable Ref      : (Mo_flg1It)
SW Sys Cond          : (i(MOCCPU_IT_SY) ==1)
SW Variable Ref. Sys Cond
SW Variable Ref      : (Mo_flg2It)
SW Sys Cond          : (i(MOCCPU_IT_SY) ==1)
SW Variable Ref. Sys Cond
SW Variable Ref      : (Mo_flg3It)
SW Sys Cond          : (i(MOCCPU_IT_SY) ==1)
SW Variable Ref. Sys Cond
SW Variable Ref      : (Mo_data00ItU8)
SW Sys Cond          : (i(MOCCPU_IT_SY) ==1)
SW Variable Ref. Sys Cond
SW Variable Ref      : (Mo_data01ItU8)
SW Sys Cond          : (i(MOCCPU_IT_SY) ==1)
SW Variable Ref. Sys Cond
SW Variable Ref      : (Mo_data02ItU8)
SW Sys Cond          : (i(MOCCPU_IT_SY) ==1)
SW Variable Ref. Sys Cond
SW Variable Ref      : (Mo_data03ItU8)
SW Sys Cond          : (i(MOCCPU_IT_SY) ==1)
SW Variable Ref. Sys Cond
SW Variable Ref      : (Mo_data04ItU8)
SW Sys Cond          : (i(MOCCPU_IT_SY) ==1)
SW Variable Ref. Sys Cond
SW Variable Ref      : (Mo_data00ItU16)
SW Sys Cond          : (i(MOCCPU_IT_SY) ==1)
SW Variable Ref. Sys Cond
SW Variable Ref      : (MoGlbl_bbItOut1)
SW Sys Cond          : (i(MOCCPU_IT_SY) ==1)
SW Variable Ref. Sys Cond
SW Variable Ref      : (Mo_flg0ItPackedBit1)
SW Sys Cond          : (i(MOCCPU_IT_SY) ==1)
SW Variable Ref. Sys Cond

```



Common PS-EC Monitoring SW Guidelines

Example: Generic Boolean interface for FIT in C-File

```
#define SET_MoI_bIrvErrTrqCmp      (Mo_flg0It = TRUE)
#define GET_MoI_bIrvErrTrqCmp      GET_Mo_flg0It
#define CLR_MoI_bIrvErrTrqCmp      (Mo_flg0It = FALSE)

#define SET_MoI_bTstTrqCmp         (Mo_flg1It = TRUE)
#define GET_MoI_bTstTrqCmp         GET_Mo_flg1It
#define CLR_MoI_bTstTrqCmp         (Mo_flg1It = FALSE)

#define B_i_ska_uc                  Mo_flg2It

#define SET_MoITrqCmp_bTstRst       (Mo_flg3It = TRUE)
#define GET_MoITrqCmp_bTstRst       GET_Mo_flg3It
#define CLR_MoITrqCmp_bTstRst       (Mo_flg3It = FALSE)

#define MoI_ctTrqCmp                 Mo_data00ItU8
#define MoI_rTrqPtdFlt               Mo_data01ItU8
#define MoI_rTrqInrAct               Mo_data02ItU8
#define MoI_stMoI                    Mo_data03ItU8
#define MoI_ctRst                     Mo_data04ItU8
#define MoI_facTrqErrSum              Mo_data00ItU16

#define MoI_rTrqPtdFltWgh_mp_Display Mo_data00ItU8_mp_Display

#define MoI_stSys                     MoGlbl_bbItOut1
#define MoI_bPreDrv_MoGet              Mo_flg0ItPackedBit1_MoGet
#define MoI_bPostDrv_MoGet              Mo_flg1ItPackedBit1_MoGet
```

Exceptions:

For CAN Arrays in MoFFrmCo, a set of generic Arrays must be created for instruction test. E.g Signed 16 Array MoFFrm_data00ItAryS16.

Furthermore don't use measurement points from Mo_Glbl, because it doesn't make sense to import measurement points into other functions.

A copy of a level 2 measurement point in the specific FC is to apply.

To define Message Arrays in Mo_Glbl for instruction test is not suitable. The target to save RAM is not fulfilled, because message arrays are very seldom.

Example: Can Array for FIT in MoFFrm_xx

```
SW Variable Ref. Sys Cond
SW Variable Ref      : (MoFFrm_data00ItAryU8)
SW Sys Cond          : ( i(MOCCPU_IT_SY) ==1)

SW Variable Ref. Sys Cond
SW Variable Ref      : (MoFFrm_data02ItAryU8)
SW Sys Cond          : (i(MOCCPU_IT_SY) ==1)

#define MoI_SplLvl2RxBuf_ST2_EM2_Stat1_AR      MoFFrm_data00ItAryU8
#define MoI_SplLvl2RxBuf_ST2_TCM_EngIntrvntn_AR MoFFrm_data02ItAryU8
```

To keep quality the same, access to generic variables for instruction test, must be done by #define.

E.g #define MoI_stMoF Mo_data00ItU8 /* MoF_stMoF */

It is not allowed to use a generic variable for instruction test more than once in one module.

Functional instruction test must be clamped with (MOCCPU_IT_SY==1)

For instruction test variables and instruction test parameters conversion formula dez is required.



22 Variant mechanisms for L2 and L3

Basic principles:

Complexity in monitoring functions (L2, L3) should surely be controllable. Reason: Error prevention

All variants must be tested with FCI.

For FCF: It has to be checked that all used variants are validated. (emFace)

Variant type A is used as before (different FC alternatives).

Variant type B is not used (FC-B and FC-P).

Variant type C can be used with restrictions, see chapter "[Conditional compiling with system constant \(variant type C\)](#)".

22.1 Code words

Support of different variants with a calibration parameter.

22.2 Variant coding

Takeover variant coding from L1 and do plausibility check. Indirect fault detection is allowed, e.g. switch to the wrong CAN protocol would lead to a checksum error.

Other examples:

Detection of onboard power supply protocol

MoFVar_Co transmission type, chassis variant, etc.

[Example: Variant coding in MoFVar_Co](#)

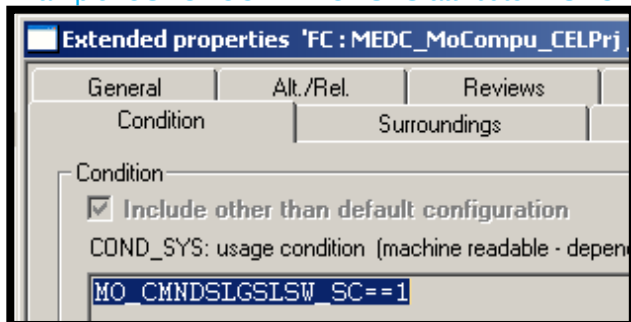
```
stCodTra_T0u8hh = AGK2EDC_stCodTra;
/* Variant: Manual Transmission */
if ((stCodTra_T0u8hh == MOFVAR_NSQ)
    ||
    (stCodTra_T0u8hh == MOFVAR_NSQ_SPORT)
    ||
    (stCodTra_T0u8hh == NSQ_KURZE_UBERSETZUNG)
    ||
    (stCodTra_T0u8hh == FSG_SPORT))
{
    /* Update the temp. variable for Gearbox Configuration */
    GearBoxTyp_T1u8hh |= MOFVAR_MANGEAR_MSK;
}
/* Variant: Automatic Transmission */
if ((stCodTra_T0u8hh == NAG_1)
    ||
    if ((
        /* Check if Power Train is Active*/
        (Tmp_bPtActyDlyBit1 == TRUE)
        &&
        /* Check if Variant change is requested */
        (GearStatus_T2u8lh == MO_TRUE)
    )
    ||
    /* Check for the complement error */
    (AGK2EDC_stCodTraCpl != (uint8)~stCodTra_T0u8hh)
    ||
    /* Check for plausibility of Gear Box coding */
    ((GearBoxTyp_T1u8hh & MOFVAR_GEARTYPE) == MO_ZERO))
```

22.3 Conditional packaging

The conditional packaging with the help of system constants can take place only at FC level. A corresponding support is available from on eMFace-Tool Version 2.1.0 E.g. conditional packaging within BC_MO : Mo / 1.6.0.

Two mechanism for conditional packaging are available, USAGECONDITIONSYS attribute in SDOM (via tab "condition" in info block of FC) and COND attribute. The first has to be specified already during development of an FC, the latter has to be specified during packaging of an FC into a GC_MO.

Example: USAGECONDITIONSYS attribute in SDOM



Example: COND attribute in SDOM

GC_MO : MoFVehic / 72.4.0; 0	INDEVE...				
FC : MoFACC_Co / 30.0.0; 6	AVAILAB...	recursive closed	(1)	(1)	(MO_CMNDGSLSW SC == 1)
FC : MoFACC_Co / 37.0.1; 1	AVAILAB...	exempt closed	(1)	(1)	(MO_CMNDGSLSW SC == 2)
FC : MoFBik_Co / 26.2.2; 1	AVAILAB...	genera... closed	(1)	(1)	(MO_CMNDGSLSW SC == 1)
FC : MoFBik_Co / 46.1.0; 3	AVAILAB...	exempt closed	(1)	(1)	(MO_CMNDGSLSW SC == 2)

22.4 Conditional compiling with system constant (variant type C)

It is allowed to use variant type C, but only if other variant types as listed above are not reasonable or cannot be used.

The following application rules are important to limit complexity of monitoring software and to allow a functional instruction test without using variant type C.

Another important point is to minimize the number of programs needed to test all Monitoring variants.

Application rules:

- Nested system constant usage should not be used.
- A switch between SW-features should not be used, whereas activating a feature (e.g. `#if (PTO_SY, STSP_SY==1) { }`) is allowed. That means, it is not allowed to use `#else/#elif/...` when in else and if are different features (switch of features). It can be used only for initialisation or interfaces (e.g. 2 analogue accelerator pedals or 2 digital accelerator pedals).

Example: Activating a feature

```
#if (CMBTYP_SY == CMBTYP_DS)
/* assignment to _rel_q6plem3_s16: min=-32768, max=32767, hex=4096/25phys+0, limit=(maxBitLength: true, assign: true)
(_rel_q6plem3_s16 = ((MoF_stVarDrvChar == 2U) ? MO_KF_P_MoFTrqPtd_facDrvDmd2_M((sint16)MoF_nClth, (sint16)MoF_rAPP)
#endif
```



If functional instruction test is used:

- All features should be included unconditionally in L2`.
- For switching between interfaces only one interface has to be considered in L2`.

Example: Switching between features

```
/* assignment to _rel_q6p1em3_s16: min=-32768, max=32767, hex=4096/25phys+0, limit=(maxBitLength: true, assign: true),
(_rel_q6p1em3_s16 = ((MoI_stVarDrvChar == 2U) ? MO_KF_P_MoITrqPtd_facDrvDmd2_M((sint16)MoI_nClth, (sint16)MoI_rAPP) :
/* assignment to _rel_q6p1em3_s16: min=-32768, max=32767, hex=4096/25phys+0, limit=(maxBitLength: true, assign: true)
```

23 Usage of level 1 system constants as value

The usage of level 1 system constants as value in L2 Code (not for conditional compiling using #if) is generally allowed.

Typically a system constant will be imported from gconf_sy_pavast.xml and the value is set during PVER configuration.

Example: Definition of array size (imported from level1)

24 Terminal 15 Turn Off/Turn On without reset

Current GS

Most GS systems trigger a Software Reset when Terminal 15 is switched on during post drive (exception Daimler).

A concept was developed for GS systems without reset in the situation, in order to emulate the old behaviour for the application software (initialization of defined memory areas to zero and Relni Task)

Current DS

DS systems trigger a Software Reset only if Mo_stlrvErrReacMsg is set when entering Postdrive.

PS-EC

The goal is to achieve a reset-free restart from Postdrive to Drive (even with irreversible errors set). Can be achieved with initialization of defined memory areas to zero and Relni Task.

Attention:

For GS reset-free systems (POSTDRV_SOFT_RST_SY==0) the interface Mo_stlrvErrReacMsg is stubbed in GS systems with 0 (SyC package is identical) and should not be written with any other value.

Hint:

Currently messages can only be allocated in NoRelni sections, therefore they have to be initialized explicitly to 0, see general PS-EC coding guidelines.



25 Measurement points

Measurement point should be implemented according to PS-EC Coding guideline.

Examples:

MoF_stStrtIn1_MP_Display (stStrtIn_mp_T8u8ll)

Former Measurement point

```
MoF_stOverlay_mp = strtErr_T3u8lh;
```

Measurement point follows PS-EC monitoring guidelines

```
MoF_stOverlay_mp_Display(strtErr_T3u8lh);
```

Definition in PaVaSt:

<SW-CLASS-REF>TYP_Mo_RamMpDispU8</SW-CLASS-REF>

See chapter "[PaVaSt creation](#)".

26 Direct hardware access

Former GS

Digital and analog input signals are encapsulated with defines in Mo_SwConf.h (outside BC_Mo).

Former DS

Direct access with Core macros to hardware, e.g. DIO_GET (PTOSwt_I_D).

PS-EC

Direct access with Core macros to hardware in related FC.

CAN buffers, Flexray Buffers, etc., e.g. Com Buffer should be defined in PaVaSt (current exception: volatile not possible within PaVaSt).

Therefore a central Mo_SwConf.h outside Monitoring package is not used anymore.

If needed a configuration header for the affected FC can be used,

E.g. SWHDR : moftra_ptgrip_conf / 1.1.0; 1

Example: FC specific configuration header

```

#include      "adc.h"                      /* for the ADC macros */

#if (CMBTYP_SY == CMBTYP_GS)              /***** gasoline system ****/

    #if (SY_DAPRID == 4)
        /* ADC */
        #define      MOFTRA_PTGRIP_GET_ADC_ZPS1      Adc_GetRawRatio(E_A_GANG1)
        #define      MOFTRA_PTGRIP_GET_ADC_ZPS2      Adc_GetRawRatio(E_A_GANG2)
    #else
        /* ADC */
        #define      MOFTRA_PTGRIP_GET_ADC_ZPS1      0
        #define      MOFTRA_PTGRIP_GET_ADC_ZPS2      0
    #endif

#else                                      /***** diesel system ****/
    /* ADC */
    #define      MOFTRA_PTGRIP_GET_ADC_ZPS1      Adc_GetRawRatio(GNS1_I_A)
    #define      MOFTRA_PTGRIP_GET_ADC_ZPS2      Adc_GetRawRatio(GNS2_I_A)
#endif

```


**Common PS-EC Monitoring SW Guidelines**

FC : MoFTra_PtGrip / 1.1.1; 1	AVAILAB...
X CONFDATA : moftra_ptgrip / 1.0.0; 0	AVAILAB...
SPEC : MoFTra_PtGrip / 1.1.0; 2	AVAILAB...
H SWHDR : moftra_ptgrip_conf / 1.1.0; 1	AVAILAB...
C SWSRC : moftra_ptgrip / 1.1.1; 1	AVAILAB...
X TDATA : moftra_ptgrip / 1.1.0; 1	AVAILAB...

27 Complement Variables

GS

All complement variables are "READ-ONLY".

DS

All complement variables are "NOT-ACCESSIBLE".

PS-EC

SW Calibration Access for complement variables according Spec.

28 Request ICO in level 1

Mo_Glbl defines and writes the new PS-EC interface Mo_flgICOMsg (Bit), the old DS interface Mo_stICOMsg(Value) and the old GS interface B_i_ska_um depending on Mo_stMoF & MO_ICOL2_MSK.

Mo_Glbl_Proc is called every 10ms as last function in MoExe_Co, but without contribution to PFC.

For MDG1 please refer to chapter Mo_Glbl. Writing of these L1 interfaces in the other L2 FC's is therefore not needed anymore.

Example: Central request ICO L1

```
void Mo_Glbl_Proc(void)
{
    /* EasyCASE ( 177
    Globale Definitionen

    ICO Anforderung an Ebene 1 */
    if ((Mo_stMoF & MO_ICOL2_MSK) == MO_ICOL2
    {
        B_i_ska_um      = TRUE;
        Mo_flgICOMsg    = TRUE;
        Mo_stICOMsg      = TRUE;
    }
    /* EasyCASE ) */
}
```



29 Functional Conventions

29.1 Debounce counters

GS: >= decision that means errors reaction cannot be deactivated

DS: > decision that means errors reaction can be deactivated

PS-EC: in one FC only one strategy should be used.

29.2 Initial data set

GS: functional calibration example

DS: neutral calibration

PS-EC: use general PS-EC guideline according process library: "New features should have a neutral calibration".

Additional guideline for monitoring: There should be no additional algorithms in order to support neutral behaviour (no error reaction).

30 Link to Checklist

MDG1

First release of migration checklist to MDG1 is available

[MDG1_MigrationChecklist.xlsx](#)

31 Validation of conversion to PS-EC Coding Guidelines Monitoring

Necessary actions:

- All changes must be checked by SW-Review.
- FCI SW Analyser log result: Full assessment , if necessary compare of log file before and after conversion
- Object comparison
- Identification of necessary functional tests (FCF) during SW Review. Creation of test specification for FCF.
- FCF is not necessary for standard changes which are already tested in other functions. For all functions the validation condition must be set and documentation must be stored in Test-F container. (Decision from SW Review)
- Overview of necessary functional tests stored in Test-F of BC_MO

[Examples for conversion to PS-EC monitoring guidelines DS and GS](#)

```
BC_MO : Mo / 1.5.8_DGS; 1
BC_MO : Mo / 1.5.8; 1
TESTCASE : mo_bci_spec / 1.5.8_DGS; 1
```

```
BC_MO : Mo / 1.7.3_DGS; 1
BC_MO : Mo / 1.7.3; 1
TESTCASE : mo_bci_spec / 1.7.3_DGS; 0
```



32 SW-Review and FCI

- If the development is done with Autocode, and the Code Generator is not classified/qualified accordingly, an implementation review for monitoring FCs always includes a SW-Review (referred to also as Code Review).
- It is recommended to execute SW-Review after FCI is finished.

33 Recommendation to avoid stack usage

Argument passing

The TriCore uses eight registers (D[4] through D[7], and A[4] through A[7]) to pass up to four non-pointer arguments and up to four pointer arguments from the caller to the called routine.

Arguments beyond those are passed on the stack. Means only up to four non-pointer arguments and up to four pointer arguments for argument passing shall be used as long as no stack write ability test is present.

34 Software Function Call Coverage

For ASIL-C and ASIL-D software functions it is mandatory (strongly recommended (++) by ISO26262) to achieve 100% function call coverage. Therefore it is mandatory to add such software functions to the program flow check, short PFC.

35 Extended monitoring task

An extended monitoring task is only allowed in MDG1.

- Mo_Tmp variables must not be used for extended task. For extended task process local variables, means registers or stack, have to be used.
Hint:
 - For ASCET autocode models the implementation of the model (Block Diagram) can be changed from "Impl" to "LVL2". With this setting ASCET will not map process local variables to Mo_Tmp variables, but keep stack variables
 - MLSL doesn't generate Mo_Tmp variables, therefore no adaptations necessary
- All level2 functions which are developed and scheduled for the extended task follow the naming convention MoFxxx_Co_Proc_<Task> or Mo..._Proc_<Task> where Task is the scheduling time (e.g. MoFTrqPtd_Eng_Proc_5ms). If function is scheduled in BSW-End part it has to be included in the name (e.g. MoFTrqPtd_Eng_Proc_5ms_BSWEnd)
- Only reentrant services from MoFSrvLib must be used in the extended task. Most services from MoFSrvLib are reentrant. For not reentrant services in MoFSrvLib an additional reentrant service is available and shall be used in the extended task.
- Mo_stMoFExtdTsk and Mo_stMoCExtdTsk have to be used in the extended task instead of Mo_stMoF and Mo_stMoC. In Mo_stMoFExtdTsk only the bit



positions for Mo_bICOL2, Mo_bAddDisbIPrelCORst and Mo_bAddConPFC shall be used. The same masks shall be used for Mo_stMoC / Mo_stMoF and Mo_stMoCExtdTsk / Mo_stMoFExtdTsk (e.g. MO_ICOL2_MSK).

- Every function of the extended task has like every other monitoring function a unique PFC ID. The PFC ID for the extended task follows the naming convention: <FC>_PFC_ID_EXTDTSK
- For program flow check (PFC) of the extended task MoCPFCExtdTsk_Co_Set(<FC>_EXTDTSK_PFC_ID) and MoCPFCExtdTsk_Co_Check() have to be used
- All functions of the extended task have to be scheduled together in a separate MoExe proc. This proc has to be scheduled in BSW-End or in ASW of the Extended Task which needs to be computed on the same core together with the 10ms task.
- The system constant MO_EXTDTSK_SC has to be set to
 - "0" if, Support for extended monitoring task disabled"
 - "<x>" if, Support for extended monitoring task in <x>ms raster "(e.g. 5 for 5ms raster)

36 Failure Modes of QM-rated services/functions

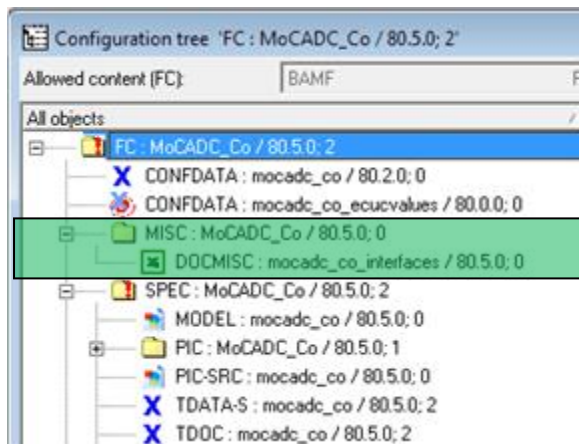
If ASIL relevant level 2/3 function use interfaces or service functions (including macros) from level 1 (QM-rated), e.g. Messages, Adc, Dio, Pwm, InjVlv, Reset etc., the following steps are mandatory:

- Analyse and evaluation of all used interfaces / service functions regarding potential impact and failure modes
- The analysis has to be reviewed and documented in at least one of the following options:
 - Excel spreadsheet as part of FC in SDOM
 - FMEA
 - "Programmers section" of FC document

The following procedure represents the best practice:

- Interfaces / services functions failure mode has to be documented in excel-template which is part of FC in SDOM:
 - Interface failure mode evaluation is standardized
 - Interface failure mode evaluation is version controlled
 - Interface failure mode evaluation is reviewed in code review

[Example for documentation of Service/function failure modes](#)

Common PS-EC Monitoring SW Guidelines**Hint:**

If a latent fault in the interface or service function (loss of functionality) leads to a safety issue like deactivated monitoring function or significant decrease of diagnostic coverage, measure must be taken to avoid error propagation: additional checks to check correct behavior of the service function, implement the service in level2, request a monitoring specific interface.

37 Usage of Pointer

It is allowed to use pointer in monitoring, but the usage of pointer shall be restricted and only used if no other appropriate method is possible. Pointer on a function e.g. is forbidden. For new services it shall be checked if an argument as a pointer is necessary or if the same behavior can be achieved by using a service with a return statement.

Pointer usage is allowed for e.g.:

- Pointer to static arrays
- Pointer to static structures
- Pointer as argument e.g. for monitoring service library and BSW function calls

38 Usage of tool MoFFrm_Nds

Projects may decide to use tool MoFFrm_Nds deployed in toolbase to automate the development of CAN monitoring modules, e.g. MoFFrm_Rx, MoFFrm_Tx and MoXFrm_Co. The tool serves as a help in reducing development effort in Design and Coding phase. The users must still adhere to all the process steps mentioned in Process library to ensure the quality of the output of the tool.

The prerequisites and details on how to use the tool are present in

[How To MoFFrm_Nds Standalone.](#)